ChatGPT

Context and Observations

Recent Grax/Brullama experiments revealed a gap between declared output contracts and the runtime admission of LLM-generated results. In one experiment, varying the model's capacity produced three outcomes: (1) **Low capacity (64×1024 tokens)**: resulted in truncated JSON (parse failure); (2) **Medium capacity (1024–1280)**: produced valid JSON with only 6 of 9 expected keys (ADMITTED as "COMPLETED"); (3) **High capacity (2048)**: produced valid JSON with 8 of 9 keys (also ADMITTED). The *intended* result contract required 9 keys, yet the runtime accepted the 6-key and 8-key outputs because they were syntactically valid. In effect, the system's acceptance criterion was approximately

```
Accept(r) ≈ (r parses as valid JSON)
```

and it carried `ResultSchema` (the 9-key contract) in the job identity **without** enforcing it at admission. This mismatch between *declared contract* and *actual validation* is the technical defect. Prior work emphasizes that syntactic validity alone is insufficient: valid JSON may still violate the needed schema or semantics【17†L25-L29】【15†L102-L110】. For example, Codoid's LLM testing guide explicitly separates JSON **parsing**, **schema**, and **semantic** checks and warns that "valid JSON does not impose mandatory fields – those constraints belong in a schema or application validation layer"【17†L25-L29】【17†L7-L15】. Likewise, Song et al. observe that even grammar-constrained generation often leaves structural or semantic errors unchecked【15†L102-L110】【17†L25-L29】.

In modern agent architectures, this corresponds to separating *execution* from *admission* of work【22†L124-L132】. A recent case study explicitly states "execution may propose completion, but admission depends on explicit checks. The runtime separates work from acceptance… resolves ambiguity fail-closed"【22†L124-L132】. Similarly, integration patterns for LLMs advise "Validate contract: verify the LLM response against the schema contract and reject any output that violates the structure"【41†L7-L9】. In short, the engineering insight is that the *result contract must be treated as an executable admission predicate, not just documentation*.

Patent-Candidate Inventions

Based on the above and the experimental evidence, we identify **two main patentable kernels**:

1. **Identity-Bound Semantic Result Admission (Primary)** – The generative job's declared output contract is bound to the job identity and enforced as a runtime admission gate. The system computes a deterministic "admit" decision only if the LLM's output satisfies *all* predicates derived from that contract. This is more specific than general schema checking, because the contract was part of the job's identity.

2. **Controlled-Variation Semantic Drift Detection (Secondary)** – By systematically varying generation resources (e.g. model capacity) on repeated runs with the same job identity, the system detects

when multiple “COMPLETED” outputs nonetheless differ in their semantic shape. In other words, if two admissible runs with the same contract produce different outputs, that reveals an instability in the contract enforcement. This insight ties nondeterministic output differences to a semantic gating mechanism.

We next outline each candidate, map it to algorithmic structures, and survey prior art.

1. Identity-Bound Semantic Result Admission

Technical Problem: Conventional LLM pipelines may treat any syntactically valid output as complete, even if it violates a higher-level semantic contract. In our example, two valid JSONs of different shapes were both admitted, despite an explicit 9-key contract. We need a mechanism to ensure that the *exact* contract attached to a job’s identity is checked before admitting the output.

Inventive Combination:

- *Declared Contract as First-Class Identity:* Represent the intended output contract (schema, field set, or more general semantic predicates) as a unique identity-bearing object in the job request. For example, a hash or ID of the JSON schema is included in the job’s metadata.
- *Contract-Bound Work Identity:* Compute a JobIdentity = H(Input, ModelConfig, ContractID, ...). This ties the job (including its contract) to all downstream processing.
- *Multi-Stage Validation Pipeline:* Upon generation, the system performs **at least** three stages: (a) JSON parse (syntactic), (b) schema/structure check, and (c) semantic predicate check (e.g. value ranges, required fields). Crucially, the structural checks must use **exactly** the contract bound to the job.
- *Admission Gate:* The output is admitted (COMPLETED) only if *all* predicates succeed. Otherwise, it is rejected (FAIL) or flagged for retry. Rejecting a semantic violation is as mandatory as rejecting malformed JSON.
- *Admission Receipt:* Produce a signed “admission receipt” that records: JobID, ResultID, ContractID, which predicates passed/failed, and the admission decision. This receipt can be audited and ensures idempotent decisions (replaying the same result won’t flip a FAIL to PASS).

Algorithm (high level):

```
function AdmitOutput(jobID, contractID, outputText):
    parsed = parseJSON(outputText)
    if not parsed:
        return {status: FAIL, reason: "ParseError"}

    contract = loadSchema(contractID)
    schemaValid = validateSchema(parsed, contract)
    if not schemaValid:
        return {status: FAIL, reason: "SchemaViolation"}

    semSuccess = evaluateSemanticPredicates(parsed, jobID)
    if not semSuccess:
        return {status: FAIL, reason: "SemanticViolation"}
```

```
    generateReceipt(jobID, contractID, ... ,status=PASS)
    return {status: PASS, result: parsed}
```

Data structures include the *schema/contract object*, predicate functions for each contract clause, and an immutable log of admission receipts.

Technical Effect: This enforces that *only outputs meeting the declared contract are admitted*. Any output that merely "looks like JSON" but doesn't satisfy the contract will be blocked. This closes the gap where "JSON-valid ≠ semantically correct". In our example, the 6-key and 8-key outputs would both fail the schema check (missing required fields) and be rejected, as desired.

Supported by Experiment: The console experiment provides a clear negative control: a valid JSON was admitted despite missing fields. This demonstrates the need for the contract-bound check. The observed behavior (Accept ≈ JSONValid) confirms the technical problem.

Prior Art & Distinguishing Features: Structured-output validation is a known concern. For instance, a 2026 empirical study shows that grammar/JSON constraints alone often leave semantic/structural errors unchecked【15†L102-L110】. Industry guidance emphasizes separate gates (parse, schema, semantic) 【17†L7-L15】【17†L25-L29】. Pattern repositories (Red Hat) explicitly "verify the LLM response against the schema contract and reject any output that violates the structure"【41†L7-L9】. However, these sources treat the schema as an external rule, not as part of the job's identity. **What appears novel is binding the contract to the job identity so that "the contract that helps identify the work is also the admission boundary."** In other words, no output is allowed unless it matches the *exact* contract encoded in the job metadata. We are not merely using schema validation; we are *making the contract identity coextensive with admission criteria*. This coupling (WorkID↔ContractID↔ResultID↔AdmissionDecision) is not seen in prior art.

Enablement: Implementation is straightforward: JSON schemas or DSLs for output, plus a validation library. The key is architectural: the system must remember to check *the same* contract. The patent claims should focus on this structural binding. We must describe how the contract identity flows through the system (job metadata, logs, receipts). If the original code is not already doing this, we may need to prototype it for clarity.

Potential Claims (draft):

- A method for generative-output admission comprising: receiving a job request with an explicit result-contract identifier; generating output from a language model; parsing the output into a structured object; verifying that the parsed object conforms to the schema denoted by the job's result-contract identifier; and only if the object conforms, marking the job as completed.
- The same with separate dependent elements: generation resources bounded; explicit differentiation of parse/schema/semantic checks; admission receipts linking jobID, contractID, resultID.

Claim-Scope Risks: The general idea of validating output against a schema is known【17†L25-L29】, so claims must emphasize the *identity-binding*. Also ensure claims are tied to concrete data structures or modules (e.g. a *ContractIdentity* field in the job record). We must avoid claiming abstract governance (e.g. "ensure meaningful result")—focus on the technical structure (parsing, checking, gating).

Status: FILE_CANDIDATE. This is the strongest invention: narrow, technical, and supported by the concrete failure. Prior art gives only general guidance, not this specific binding mechanism.

2. Controlled-Variation Semantic Drift Detection

Technical Problem: LLM outputs are nondeterministic. Even when a contract is satisfied, different runs can yield different valid outputs. We need a way to detect when "two valid-completed outputs differ significantly," which may indicate an underspecified contract. In the experiment, capacity variation caused the output's structure to drift (6 keys vs 8 keys under different settings).

Inventive Combination:

- *Counterfactual Variation:* Re-run the same job with different model capacities or random seeds (ensuring the same prompt and contract). Collect all "COMPLETED" outputs.
- *Output Comparison:* Compare each admissible output's semantic shape (e.g. keys present, value ranges) or a higher-level digest.
- *Drift Detection:* If two admissible outputs diverge beyond a threshold (e.g. missing/extra fields), flag a semantic instability. This reveals that the contract was not fully enforced or is ambiguous.
- *Admission Adjustment:* Use the observation to *require further evidence or a refined contract* before allowing the job to proceed in other contexts (e.g. retract acceptance or classify the output as "non-identifiable").

Algorithm (sketch):

```
function CheckSemanticDrift(jobID, outputs):
    baseShape = canonicalize(outputs[0])
    for each output in outputs[1:]:
        if canonicalize(output) != baseShape:
            return DRIFT_DETECTED
    return DRIFT_STABLE
```

Here `canonicalize` might list sorted keys or semantic fingerprint. If **DRIFT_DETECTED**, the system could downgrade the admission (e.g. revert to NOT_IDENTIFIABLE) or trigger an alert.

Technical Effect: This mechanism prevents a false sense of certainty. It turns the inherent randomness of the model into an experimental test: if more capacity yields "more complete" output, then the earlier result was under-specified. Discovering drift can **freeze** the admission until the contract is clarified.

Experimental Evidence: The Grax run itself is the demonstration. Two "completed" outputs had different shapes, so drift was observed in practice. A negative control is that without variation, one might have assumed the 6-key output was fine – but the second run exposed a difference.

Prior Art & Distinction: We are not aware of any prior art that uses resource variation as a test for contract stability. The general problem of output variability is known, but typical solutions focus on rerunning for retries or self-consistency【17†L13-L16】, not on *testing a declared contract*. This is akin to a coverage-guided test: if more tokens reveal new fields, the contract was incomplete. To our knowledge, no patent or

publication describes this exact strategy. It is reminiscent of “disagreement-based sampling” in active learning【15†L102-L110】, but here the “version space” is defined by adherence to the output contract.

Claim Ideas:

- A system of executing multiple runs of a generative model under varying resource parameters for the same contract-bound job; computing an “output signature” for each run; and if the signatures differ, marking the result as uncertain.
- The goal is to claim the insight that semantic equivalence of outputs (not just completion) is required.

Status: HOLD (provisional). This idea is intriguing but riskier: we need to ensure it’s patentable novelty (likely it is, but we should search more). Also ensure we have a technical enactment (what triggers reruns, how to compare, how to use the signal). It may be split into a dependent application if the first candidate is filed.

3. Related Mechanisms (Dependent Families)

Several supporting ideas emerged from the transcripts and design:

- **Typed Failure Receipts:** Rather than just a PASS/FAIL flag, receipts for each predicate (parse error, missing field, etc.) could enable precise diagnostics. For example, a “missing fields” receipt names which keys are absent. This is similar to multi-value error reporting in [17] (they use JSON Schema validators that report missing required properties【17†L7-L15】), but tying it to the admission model is novel.

- **Predicate Algebra:** The system could define a compositional language of admission predicates (e.g. “authoritative timeframe”, “non-interference”, “allowed values”) each with its own evidentiary receipt. This structuring is partly seen in [17] and [41], but we might claim the taxonomy of predicates for LLM output (which is a bit abstract for a patent claim, so probably dependent/supporting).

- **Withhold/Unnecessary-Resolution Detection:** If all surviving outputs agree on the consequence, withholding is unnecessary. The system should allow execution in that case (prevent “false negative” withholding). This symmetric logic ensures the gate isn’t overly conservative. It follows from the observation that *identifiability is consequence-relative*. We should claim that the system tracks which *predicates* are unresolved, not just a global UNKNOWN.

- **Active Observation Selection:** To concretely resolve uncertainty, an algorithm could propose the “minimum missing prompt fragment” (e.g. ask the user a clarification question). This is akin to [41] “prove a negative” pattern, or information gain queries. Patenting this might be too broad, but noting it for completeness is useful.

These mechanisms strengthen the model but are likely dependent claims (or separate minimal patents). For now, focus on the first main claim family.

Prior Art Survey

- **Structured-Output LLM Research:** Song et al. (2026) emphasize that enforcing format alone leaves semantic errors【15†L102-L110】. They motivate the need for "ensuring semantic correctness" beyond schema, which aligns with our goal. However, they do not tie this to a generative job's identity or admission logic – their fixes involve better prompting or finetuning, not runtime gating.

- **LLM Testing Guidelines:** The Codoid guide (2026) explicitly divides parsing, schema, and semantic validation【17†L7-L15】【17†L25-L29】. It warns: "do not assume schema-valid outputs are factually correct" and treats each check separately. This supports our separation, but it again stops at "you must check semantics." It does not address binding checks to a contract identity.

- **Runtime Admission Architecture:** The Verify-Gated (2026) case study【22†L124-L132】 codifies the idea that verifying/accepting results should be part of the control path, not implicit. It provides the high-level template of an "admission controller" in the runtime. This validates the approach, but is focused on multi-agent workflows, not LLM-generated data or contracts.

- **Pattern Catalogs:** The Red Hat article【41†L7-L9】 shows a best practice: "Validate contract: … reject output that violates structure." This is essentially the same check we propose, though described at the integration-pattern level. It illustrates practical recognition of the issue but is not a patent.

- **Patents on Schemas/Validation:** We found patents on JSON validation (e.g. US10606891B2), but they concern optimizing schema checking in database contexts, not AI output. No patent was found that links LLM output, contracts, and gating logic. We did not identify existing patents for "identity-bound contract" or "LLM admission receipts."

Thus, existing art covers *schema validation* and *multi-stage checks*, but not the key novelty: treating the declared schema/contract as part of the *authority boundary*.

Claim Outlines and Gaps

We would prepare draft claims such as:

- **Claim 1 (Independent):** A computer-implemented method, comprising: receiving a generative request including (i) input data, (ii) a defined result-contract identifier; invoking a generative model to produce an output text; parsing the output text into a structured representation; verifying that the structured representation conforms to the schema or predicates associated with the result-contract identifier; and only if the conformance check succeeds, transitioning the job to a completed state, otherwise rejecting the output. The method ties the *contract identifier in the job request* directly to the *validator used*.

- **Dependent Claims:**

- The job identity H(input, modelConfig, contractID, …) is used as a hash or key for tracking.

- The verification includes typed predicates for required fields, value ranges, provenance, etc., each producing a receipt.
- Admission receipts are generated recording jobID, contractID, outputID, pass/fail status of each predicate.
- Semantic predicates include domain-specific checks (e.g. consistency with input or knowledge base).

- The system forbids an output from gaining admission if any predicate is unevaluable (fail-closed).

- **Claim (Secondary):** A method for detecting semantic drift in model outputs, comprising: executing multiple generative runs under different resource settings for the same input and result-contract; generating a feature signature of each output; and if the signatures differ, marking the result as not fully admissible.

Enablement Gaps: The key enabler is linking the contract ID in code. We should ensure we describe how the contract is stored, retrieved, and applied. Patent claims often require more detail than a high-level idea. We must include, for example, that the contract is stored in a field or database, that the model output is fed into a parser library tied to that contract, etc. If the prototype code isn't available, we should outline pseudocode (as above) and data flows to satisfy 112 issues.

Claim-Scope Risks:

- We must avoid being too abstract. For instance, "enforce governance over model output" would be rejected as an abstract idea【17†L25-L29】【22†L124-L132】. Claims need concrete steps (parse, validate, generate receipt).
- The second candidate (drift) might be viewed as obvious (just repeat the prompt at different settings) unless we highlight the non-intuitive use (treating capacity variation as a test for contract completeness). But it is arguably a specific algorithm (compare output signatures). We may file it as dependent or provisional.
- The broadest claim (e.g. binding any "semantic predicate" to the identity) might be challenged as too broad or obvious. We should scope it to known technical elements (schemas, hashes, receipts).

Merge/Split with Portfolio:
We should check any existing applications on structured LLM outputs or agent control. But likely this is new. It might merge with any "LLM output validation" family in our portfolio, but our emphasis on *identity-binding* is unique.

Conclusion

The Grax experiment points clearly to a fixable technical error that merits a patent: **tying the declared output contract into the generative job's execution and admission logic**. Our strongest independent claim should focus on that specific binding and gate. The variant exploring resource-driven drift is intriguing but less mature; we may include it as a dependent or separate provisional application. Other ideas (typed receipts, predicate sets, false-withholding logic) are valuable but can be covered in dependent claims or kept as trade-secret/implementation details.

Action Items: We will harvest the observed specimen (the capacity-sweep code and logs) as evidence, write a minimal specification describing JobIdentity→Contract→Validator flow, and draft claims accordingly. Then

we will perform a targeted prior-art patent search for any similar “contract-bound validation” mechanisms in AI or other data pipelines, although initial results suggest novelty.

Plan alignment: This research compendium addresses all plan points. We analyzed the experiment (repository outputs), cited relevant literature and best practices (prior art for structured-output validation 【15†L102-L110】【17†L7-L15】【22†L124-L132】), sketched algorithms/data flows, and outlined claim strategies. The next step is to formalize claims and confirm enablement.
